The Incident Response Playbook

A Practical Framework for Handling Production Incidents Under Pressure

---

Introduction

Production incidents rarely happen at convenient times.

They occur:

- in the middle of deployments
- during peak traffic
- when half the team is offline
- or during seemingly quiet moments in the day

When an incident begins, engineers are suddenly confronted with a flood of signals:

- monitoring alerts
- dashboards turning red
- logs filling with errors
- customers reporting failures

In these situations, the biggest challenge is not usually **lack of data**.

The real challenge is **lack of structure**.

Without a structured response process, teams often:

• investigate the wrong component
• pursue multiple theories simultaneously
• duplicate debugging work
• waste valuable time

High-performing engineering organisations approach incidents differently.

They follow a **repeatable incident response framework**.

This playbook introduces that framework.

Why Incident Response Needs Structure

During incidents, engineers operate under cognitive pressure.

Stress, urgency and incomplete information make decision-making difficult.

Without a structured approach, teams frequently encounter these problems:

Symptom-Driven Debugging

Teams assume the first visible error explains the failure.

Example:

A payment API returns **502 errors**, so engineers assume the gateway is broken.

Later they discover the gateway was simply reporting a **downstream database failure**.

Multiple Competing Hypotheses

Different engineers investigate different components:

- one checks networking
- another inspects databases
- another inspects authentication

This creates confusion and delays.

Investigation Without Coordination

Without clear roles, incident response becomes chaotic.

People interrupt each other, repeat the same checks, and miss critical signals.

The purpose of this playbook is to introduce **a clear and repeatable incident workflow**.

The 5 Phases of Incident Response

Effective incident handling usually follows five phases:

1 Detection
2 Stabilisation
3 Investigation
4 Resolution
5 Learning

Each phase has a different goal.

Understanding these phases prevents teams from **jumping ahead prematurely**.

Phase 1 — Detection

The incident response process begins when a signal indicates something is wrong.

Signals may come from:

- monitoring alerts
- automated anomaly detection
- customer reports
- internal engineers
- synthetic monitoring systems

Examples of early signals include:

• sudden increase in error rates
• unusual latency spikes
• authentication failures
• API timeout errors
• service health checks failing

First Questions to Ask

At this stage, responders should answer three basic questions.

Is the signal real?

Sometimes alerts are false positives.

Check:

- recent monitoring changes
- alert configuration thresholds
- recent deployments

Confirming the signal prevents unnecessary incident escalations.

What is the impact?

Understanding impact determines the priority.

Questions to ask:

- Which services are affected?
- How many users are impacted?

- Is revenue affected?
- Are critical workflows failing?

This determines incident severity.

Is the issue ongoing?

Determine whether the issue is:

• actively worsening
• stabilising
• intermittent
• already resolved

This informs how aggressively the team should respond.

Phase 2 — Stabilisation

Before attempting deep root cause analysis, responders should focus on **system stability**.

The goal is simple:

Stop the system from getting worse.

Common Stabilisation Actions

Examples include:

Traffic Reduction

- rate limiting
- load shedding
- disabling non-essential features

Rollback

If the incident began after a deployment, rollback may restore stability quickly.

Scaling Resources

Temporary scaling may relieve pressure on:

- databases
- worker queues
- compute resources

Circuit Breakers

Prevent failing dependencies from cascading failures throughout the system.

Stabilisation does not solve the root cause.

It simply **buys time for investigation**.

Phase 3 — Investigation

Once systems are stable, engineers can begin deeper analysis.

This stage focuses on identifying:

• the failing component
• the triggering event
• the underlying root cause

The Incident Investigation Checklist

Experienced responders typically investigate using a structured checklist.

1 What changed recently?

Many incidents are triggered by change.

Check:

- deployments
- infrastructure modifications
- configuration changes
- dependency updates

If something changed shortly before the incident began, it is a strong suspect.

---

2 Which systems are failing?

Identify affected services.

Example:

If these services fail simultaneously:

- payment API
- order service
- billing system

The common dependency might be:

the database

---

3 What systems are still healthy?

Healthy systems provide important clues.

For example:

If reads succeed but writes fail:

The issue may involve **database locking or replication**.

---

4 What do logs reveal?

Logs often provide the earliest evidence of failure.

Look for:

- recurring error messages
- connection failures
- timeout patterns
- unusual latency

---

5 What does the timeline reveal?

Mapping events chronologically can reveal causal relationships.

Example timeline:

Time	Event
10:01	Deployment begins
10:04	Error rate increases
10:06	Database latency spikes
10:09	Service crashes begin

This timeline strongly suggests the deployment introduced the failure.

Phase 4 — Resolution

Once the root cause is identified, engineers can implement a permanent fix.

Common resolutions include:

• fixing code defects
• correcting configuration errors
• scaling infrastructure
• replacing failing dependencies

After applying the fix, responders should verify:

- error rates have returned to normal
- latency has stabilised
- system health checks are passing

Monitoring systems should remain active for some time to ensure stability.

Phase 5 — Learning

The final phase is often overlooked.

However, it is one of the most valuable.

This phase answers the question:

How do we prevent this incident from happening again?

Conducting a Blameless Postmortem

The goal of a postmortem is learning.

Not blame.

Effective postmortems examine:

• technical causes
• process weaknesses
• missing monitoring
• documentation gaps

Key Questions for Postmortems

What allowed the incident to occur?

Examples:

- missing monitoring alerts
- insufficient testing
- poor dependency visibility

What slowed down the response?

Examples:

- unclear ownership
- poor documentation
- difficult log access

What improvements should be made?

Examples:

- improved monitoring
- stronger deployment safeguards
- better runbooks
- clearer incident ownership

Incident Response Roles

During major incidents, assigning roles can dramatically improve coordination.

Typical roles include:

Incident Commander

Responsible for coordinating the response.

They ensure:

- communication remains clear
- tasks are assigned
- investigation stays organised

Lead Investigator

Focuses on identifying the root cause.

This engineer performs the primary debugging work.

Communications Lead

Provides updates to:

- stakeholders
- support teams
- leadership

This prevents investigators from being distracted.

Common Incident Patterns

Many production incidents follow recurring patterns.

Examples include:

Dependency Failures

External APIs or databases become unavailable.

Resource Saturation

CPU, memory, or thread pools reach capacity.

Configuration Errors

Incorrect settings introduced during deployments.

Traffic Surges

Unexpected user activity overwhelms systems.

Cascading Failures

One failing service causes multiple downstream failures.

Recognising these patterns dramatically accelerates investigation.

The Role of Observability

Modern observability tools provide powerful signals:

- logs
- metrics
- traces

However, these tools still require engineers to **interpret signals correctly**.

Many teams have excellent observability but still struggle during incidents because they lack:

structured incident response practices.

Final Thoughts

Production incidents are unavoidable in complex systems.

But they do not have to be chaotic.

By adopting a structured approach, teams can:

• respond faster
• investigate more effectively
• reduce downtime
• learn from failures

The best engineering teams treat incident response as a **discipline**, not an improvisation.

This playbook provides a simple starting point.

Bonus: The 10-Minute Incident Triage Checklist

When an incident begins, start here.

1 Confirm the signal is real
2 Determine impact
3 Identify recent changes
4 Check shared dependencies
5 Look for error patterns in logs
6 Identify what systems remain healthy
7 Stabilise the system if possible
8 Assign investigation roles
9 Map the timeline of events
10 Implement a permanent fix